Assignment 3

1

How does a branch-target buffer (BTB) help in reducing the branch penalty?

> ✓ **Answer**
> By caching the targets of branched, it reduces the amount of lookup calculations required as it is already computed.

2

What is the main difference between a trap (exception) and an interrupt?

> ✓ **Answer**
> Exceptions are internal to the CPU and happen as a result to the instruction itself happening, interrupts happen external to the CPU and typically comes from other hardware as a result of something external happening, requiring the CPU to handle that situation.

3

Refer to the following two instructions:

Instruction 1	Instruction 2
`BEQZ $S0, Label`	`LD $S0,0($S1)`

a

Which exceptions can each of these instructions trigger? For each of these exceptions, specify the pipeline stage in which it is detected

> ✓ **Answer**
> BEQZ could trigger a TLB or page fault if the instruction itself was not already ready (IF).
>
> LD could trigger a TLB or page fault similarly if the instruction was not cached (IF), as well as at the MEM stage of the cycle if the memory address is not cached.

b

If the second instruction is fetched immediately after the first instruction, describe what happens in the pipeline when the first instruction causes the first exception you listed in Part A. Show the pipeline execution diagram from the time the first instruction is fetched until the time the first instruction of the exception handler is completed.

✓ **Answer**

The instruction is cancelled and a NOP is executed in its place, then the instruction address +4 is loaded into the EPC for the exception handler to deal with. Then we jump immediately to the exception handler to fix any issues, including reloading the cache with the necessary values. Once that is done, we typically are jumped back to where the instruction was in hopes to continue execution. As a byproduct, the LD instruction would also be cancelled to prevent any undesired state to reach the registers, even if it is already executing.

4

The speed of the memory system affects the designer's decision on the size of the cache block. Which of the following cache designer guidelines are generally valid? (More than one correct answer)

1. The shorter the memory latency, the smaller the cache block
2. The shorter the memory latency, the larger the cache block
3. The higher the memory bandwidth, the smaller the cache block
4. The higher the memory bandwidth, the larger the cache block

✓ **Answer**

1 and 4, since smaller cache blocks can be accessed faster, while larger cache blocks require more bandwidth to reduce latency.

5

Which of the following is generally true about a design with multiple levels of caches? (One correct answer)

1. First-level caches are more concerned about hit time, and second-level caches are more concerned about miss rate.
2. First-level caches are more concerned about miss rate, and second-level caches are more concerned about hit time.

✓ **Answer**

1, first caches need to be able to respond quickly in order to keep the average cycle speed high. The second cache can get away with more latency, but should prioritize being able to fulfill any cache request in order to reduce operations sent out to system memory or swap.

6

Which of the following statements (if any) are generally true? (One correct answer)

1. There is no way to reduce compulsory misses.
2. Fully associative caches have no conflict misses.
3. In reducing misses, associativity is more important than capacity.

> ✓ **Answer**
>
> 2, fully associative caches have no conflict misses. Since associative caches only evict cache values when the cache is completely full, we do not have "conflict" misses as values are not locked to specific locations within the cache.
>
> Compulsory misses can be mitigated through precaching, and associativity typically is less important than capacity as associativity is hardware expensive while capacity has a large impact on how much gets evicted from a cache.

7

For a direct-mapped cache design with a 32-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
31-10	9-5	4-0

a

What is the cache block size (in words)?

> ✓ **Answer**
>
> Since the offset section is 5 bits, the block size would be 32 bytes, which is equivalent to 8 words.

b

How many entries does the cache have?

> ✓ **Answer**
>
> Since the index is 5 bits as well, there would be 32 entries within the cache.

c

Starting from powering on the computer, the following byte-addressed cache references are recorded.

Address	Tag	Index	Offset	In cache?
0	00	00000	00000	No
4	00	00000	00100	Yes
16	00	00000	10000	Yes
132	00	00100	00100	No
160	00	00101	00000	No
1024	01	00000	00000	Invalid
30	00	00000	11110	Invalid
140	00	00100	01100	Yes
3100	11	00000	11100	Invalid
180	00	00101	10100	Yes
2180	10	00100	00100	Invalid

How many blocks are replaced?

✓ **Answer**

Since the cache is direct mapped and not associative, we can map each address into directly what their associated cache index is, which leads to 4 replacements in the cache, with 4 cache hits total.

d

What is the hit ratio?

✓ **Answer**

Out of 11 access, 4 were hits, leading to a ration of 4/11.

8

Increasing associativity requires more comparators and more tag bits per cache block. Assuming a cache of 4096 blocks, a 4-word block size, and a 32-bit address, find the total number of sets and the total number of tag bits for caches that are direct mapped, two-way and four-way set associative, and fully associative.

✓ **Answer**

For a direct mapped cache, for 4096 blocks, we need 12 bits of addressing for the index. Since we have a 4 word block size, we need 4 bits of offset. This means we have 4096 sets and 16 bits of tag, meaning we have 65536 bits of tag total.

For every way we increase the associativity, we decrease our sets, thus decreasing our index size and increasing the tag size.

For a 2-way cache, we would have 2048 sets and 17 bits of tag per entry for a total of 69632 bits of tag.

For a 4-way cache, we would have 1024 sets and 18 bits of tag per entry for a total of 73728 bits of tag.

For a fully associative cache, we would have 1 set and 28 bits of tag per entry for a total of 114688 bits of tag.